

T2 — Processamento Paralelo: Multiplicação de Matrizes com MPI

FPPD — Fundamentos de Processamento Paralelo e Distribuído (98713-04)

Escola Politécnica — PUCRS — 2026/1

Objetivo

Implementar a paralelização de uma aplicação computacionalmente intensiva utilizando **MPI em Go**, executá-la no cluster Atlântica variando o número de processos e nós, e realizar uma **análise de desempenho** com cálculo de speedup e eficiência.

Descrição do Problema

A **multiplicação de matrizes** é uma operação fundamental em computação científica. Dadas duas matrizes quadradas A e B de dimensão $N \times N$, a matriz resultado $C = A \times B$ é calculada como:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

A complexidade computacional é $O(N^3)$. Para $N = 3000$, são **27 bilhões** de operações aritméticas, resultando em um tempo de execução sequencial da ordem de **vários minutos** — tornando a aceleração por paralelismo claramente perceptível.

O que deve ser implementado

1. Versão sequencial (baseline)

Um programa em Go que:

- Gera as matrizes A e B com valores aleatórios (seed fixa para reprodutibilidade).
- Calcula $C = A \times B$ usando o algoritmo ingênuo (triplo loop).
- Mede e imprime o tempo de execução.
- Imprime valores de verificação (ex.: cantos da matriz C e/ou checksum).

2. Versão paralela com MPI

Um programa em Go usando o pacote github.com/mnlphlp/gompi que distribua o cálculo da multiplicação entre múltiplos processos MPI.

O grupo deve:

1. **Escolher um modelo de paralelismo** (Mestre-Escravo, Fases Paralelas, ou outro) e **justificar** a escolha no relatório.
2. **Definir a estratégia de decomposição** dos dados: como as matrizes (ou partes delas) são distribuídas entre os processos.

3. **Implementar a comunicação** necessária entre os processos para que o resultado final seja correto.
4. **Medir o tempo total** da computação paralela (distribuição + cálculo + coleta de resultados).

A versão paralela deve produzir **os mesmos resultados** que a versão sequencial (mesma seed, mesmas matrizes, mesma saída de verificação).

Experimentação no Cluster

Executar a versão paralela variando sistematicamente o número de processos e de nós, e registrar o tempo de execução para cada configuração. O objetivo é explorar diferentes combinações para compreender como o desempenho é afetado por três fatores: **(a)** o grau de paralelismo (número de processos), **(b)** a comunicação via rede (processos em nós distintos) e **(c)** o uso de hyperthreading.

Limites do cluster

O cluster impõe os seguintes limites por job:

Nós solicitados	Walltime máximo
1	16:00:00
2	08:00:00
4	04:00:00
8	02:00:00
16	01:00:00

Máximo de 2 jobs na fila por usuário.

Planejamento dos experimentos

O grupo deve definir um conjunto de configurações que permita analisar os três fatores abaixo. São necessárias **pelo menos 8 configurações distintas**, incluindo a execução sequencial como baseline.

Fator 1 — Escalabilidade: variar o número de processos (ex.: 1, 2, 4, 8, 16, ...) para observar como o speedup evolui.

Fator 2 — Comunicação via rede: para um mesmo número de processos, comparar a execução em **um único nó** versus a execução **distribuída em múltiplos nós**. Essa comparação isola o impacto da latência e banda de rede na comunicação MPI.

Fator 3 — Hyperthreading: cada nó do cluster possui um número limitado de cores físicos. Ao alocar mais processos do que cores físicos em um nó, o sistema operacional utiliza hyperthreading. Comparar o desempenho com um número de processos **igual** ao de cores físicos versus um número **superior** (oversubscription com hyperthreads) permite avaliar se o hyperthreading traz ganho real para esta aplicação.

Para cada configuração, executar **pelo menos 3 vezes** e registrar a **mediana** dos tempos.

Tamanho do problema

Usar $N = 3000$ como valor padrão. Se o tempo sequencial no cluster for inferior a 3 minutos, aumentar para $N = 4000$. Documentar o valor de N escolhido e o tempo sequencial obtido.

Análise de Desempenho

O relatório deve conter os seguintes itens de análise:

Tabela de resultados

Para cada configuração testada, registrar:

Nós	Processos	T_p (mediana)	Speedup ($S_p = T_s/T_p$)	Eficiência ($E = S_p/P$)	Obs.
1	1 (seq.)	...	1,00	100%	Baseline
...	...	...	...	...	...

Gráficos (obrigatórios)

1. **Speedup vs. Número de processos** — incluindo a reta do speedup ideal ($S_p = P$) como referência.
2. **Eficiência vs. Número de processos** — incluindo a reta da eficiência ideal ($E = 1$) como referência.
3. **Comparação intra-nó vs. inter-nós** — para um mesmo número de processos, mostrar a diferença de tempo ao executar em 1 nó versus múltiplos nós.

Discussão (obrigatória)

Responder às seguintes perguntas:

1. O speedup obtido é sub-linear, linear ou super-linear? Por quê?
2. A partir de quantos processos a eficiência começa a cair significativamente? Qual a causa provável?
3. **Impacto da rede:** ao comparar execuções com o mesmo número de processos em 1 nó versus múltiplos nós, qual é a diferença de desempenho? O que isso revela sobre o custo da comunicação via rede em relação à comunicação intra-nó?
4. **Impacto do hyperthreading:** ao usar mais processos do que cores físicos em um nó, o desempenho continua melhorando? O hyperthreading é vantajoso para esta aplicação? Justifique.
5. Usando a Lei de Amdahl e os dados obtidos, **estime a fração paralelizável (P)** da aplicação.

Entrega

O grupo deve entregar **via Moodle** um arquivo `.zip` ou link para repositório Git contendo:

1. **Código-fonte em Go**, organizado em pastas (`sequencial/`, `paralelo/`), incluindo `go.mod`.

2. **Relatório** (PDF, 3–5 páginas) contendo:

- Modelo de paralelismo escolhido e justificativa da escolha.
- Descrição da solução paralela (como os dados são distribuídos, comunicados e coletados).
- Tabela de resultados com tempos, speedup e eficiência.
- Gráficos de speedup e eficiência.
- Discussão respondendo às perguntas acima.

3. **Instruções de compilação e execução** no cluster (pode ser um `README.md` ou seção do relatório).

Apresentações: conforme cronograma da disciplina. Demonstrar os resultados obtidos, apresentar os gráficos e responder a perguntas sobre a implementação e análise.

Grupos: até 4 alunos.

Critérios de Avaliação

Critério	Peso
Corretude: versão paralela produz mesmos resultados que a sequencial	20%
Implementação paralela com MPI e justificativa do modelo escolhido	20%
Experimentação: execução no cluster com variação de processos e nós	20%
Análise de desempenho: tabela, gráficos, cálculo de speedup e eficiência	25%
Apresentação e capacidade de responder a perguntas	15%

Dicas Práticas

Ambiente no cluster

Consultar o repositório de referência `lad-go-mpi` para configuração do ambiente, compilação e execução de programas Go com MPI no cluster Atlântica.

Representação das matrizes

Usar slices unidimensionais (row-major) para facilitar o envio via MPI:

```
A := make([]float64, N*N)
// Acesso: A[i*N + j] equivale a A[i][j]
```

Divisão de trabalho

A decomposição mais natural para multiplicação de matrizes é por **linhas**: cada processo calcula um subconjunto de linhas de C . Se N não for divisível pelo número de processos, o último processo pode receber as linhas restantes.

Verificação de corretude

Ambas as versões devem usar a **mesma seed** (ex.: 42) para gerar as matrizes. Ao final, comparar os valores nos cantos da matriz C — devem ser idênticos.

Medição de tempo

Usar `time.Now()` e `time.Since()` em Go. Medir **apenas** o tempo da computação paralela (incluindo distribuição e coleta de dados, se houver). Excluir a geração das matrizes e a impressão dos resultados.

Regras Gerais

1. **Linguagem:** Go, com MPI via github.com/mnlphlp/gompi.
2. **Cluster:** Todas as medições devem ser feitas no cluster Atlântica usando SLURM.
3. **Reprodutibilidade:** Seed fixa. Os resultados numéricos devem ser idênticos entre versão sequencial e paralela.
4. **Uso de IA:** Permitido para auxílio na compreensão de conceitos e depuração. O grupo deve ser capaz de **explicar cada linha do código** durante a apresentação.
5. **Plágio:** Soluções copiadas integralmente de outros grupos resultam em nota zero.